

Міністерство освіти і науки України
НТУУ «Київський політехнічний інститут ім. Ігоря Сікорського»
Навчально-науковий Фізико-технічний інститут

КОМП'ЮТЕРНИЙ ПРАКТИКУМ 3.

Реалізація індексів

Виконала:
Студентка 3 курсу НН ФТІ групи ФБ-34
Царик Єлизавета

Реалізуємо індекси за допомогою бінарних дерев. Зберігаємо як словник.

```
class Node:
    def __init__(self, key):
        self.key = key
        self.row_ids = []
        self.left = None
        self.right = None

class Binary_Tree:
    def __init__(self):
        self.root = None

    def insert(self, key, row_id):
        self.root = self._insert(self.root, key, row_id)

    def _insert(self, node, key, row_id):
        if node is None:
            n = Node(key)
            n.row_ids.append(row_id)
            return n
        if key < node.key:
            node.left = self._insert(node.left, key, row_id)
        elif key > node.key:
            node.right = self._insert(node.right, key, row_id)
        else:
            node.row_ids.append(row_id)
            return node

    def search(self, key):
        return self._search(self.root, key)

    def _search(self, node, key):
        if node is None:
            return []
        if key == node.key:
            return node.row_ids
        elif key < node.key:
            return self._search(node.left, key)
        else:
            return self._search(node.right, key)

    def print_tree(self, node=None, level=0, prefix="Root: "):
        if node is None:
            node = self.root
        if node.right:
            self.print_tree(node.right, level + 1, "R----")
        print("    " * level + prefix + str(node.key))
        if node.left:
            self.print_tree(node.left, level + 1, "L----")
```

```
start = True;
i = 0;
tables = {};
values = {};
need_index=[];
indexes = {};
```

Функція CREATE

```

#CREATE students (name [INDEXED], group);
def create(c):
    is_var1_correct = is_correct(c[1])
    if not is_var1_correct:
        print('ERROR: variable entered incorrectly.')
        return;
    global tables
    if c[1] in tables:
        print(f'ERROR: cannot create "{c[1]}" again.')
        return;
    columns = words_inside_brackets(c)
    if "INDEXED" in columns:
        search = True;
        global need_index;
        while search:
            find_indexed = columns.index("INDEXED")
            if columns[find_indexed-1]:
                need_index.append(f"{c[1]}_{columns[find_indexed-1]}")
            del columns[find_indexed]
            if "INDEXED" not in columns: search = False;
    tables[c[1]]=columns
    print(f'Table "{c[1]}" was created.')
    return

```

Функція INSERT

```

#INSERT INTO owners("1", "Vasya", "30");
#INSERT cats ("10", "1", "Murzik");
def insert(c, is_into):
    if is_into:
        table_name = c[2]
    else:
        table_name = c[1]
    values_in = words_inside_brackets(c);
    global tables
    global values
    global i
    global need_index
    global indexes
    if table_name in tables:
        if len(values_in) == len(tables[table_name]):
            table_name_i = f"{table_name}_{i}"
            values[table_name_i] = values_in;
            i=i+1;
            print(f'Values were inserted into "{table_name}"')
            for col_index, col_name in enumerate(tables[table_name]):
                index_name = f"{table_name}_{col_name}"
                if index_name in need_index:
                    if index_name not in indexes:
                        indexes[index_name] = Binary_Tree()
                        key = values_in[col_index]
                        indexes[index_name].insert(key, table_name_i)
                        #indexes[idx_name].print_tree();
            return
        else:
            print('ERROR: number columns != number values.')
            return;
    else:
        print(f'ERROR: no table with name "{table_name}"')
        return

```

```

def get_rows(table_name):
    rows = []
    for key, val in values.items():
        if key.startswith(f"{table_name}_"):
            rows.append(val)
    return rows

def check_table(table_name):
    if table_name not in tables:
        print(f'ERROR: no table with name "{table_name}"')
        return False;
    return True;

def print_table(table_name, table_rows):
    if (table_name == None) or (table_rows == None):
        print('ERROR: Columns or rows are not found.')
        return
    print(table_name)
    print('-----')
    for row in table_rows:
        print(row)

```

```

def join_on(table_name_1, table_name_2, on, column_1, column_2):
    if (check_table(table_name_1)) and (check_table(table_name_2)):
        rows1 = get_rows(table_name_1)
        rows2 = get_rows(table_name_2)
        cols1 = tables[table_name_1]
        cols2 = tables[table_name_2]

        if not on:
            joined_rows = []
            for r1 in rows1:
                for r2 in rows2:
                    joined_rows.append(r1 + r2)
            joined_cols = cols1 + cols2
            return joined_cols, joined_rows

        if column_1 not in cols1 or column_2 not in cols2:
            print("ERROR: column name not found.")
            return None, None

        i1 = cols1.index(column_1)
        i2 = cols2.index(column_2)

        index_name = f"{table_name_2}_{column_2}"
        global indexes

        if index_name in indexes:
            print(f"Index used for JOIN: {index_name}")

            tree = indexes[index_name]
            joined_rows = []

            for r1 in rows1:
                key = r1[i1]
                matching_ids = tree.search(key)
                for rid in matching_ids:
                    if rid in values:
                        r2 = values[rid]
                        joined_rows.append(r1 + r2)

            joined_cols = cols1 + cols2
            return joined_cols, joined_rows

        joined_rows = []

        for r1 in rows1:
            for r2 in rows2:
                if r1[i1] == r2[i2]:
                    joined_rows.append(r1 + r2)
            joined_cols = cols1 + cols2
            return joined_cols, joined_rows
    else:
        return None, None

```

```

def where(columns, rows, col, op, val):
    global indexes
    global tables
    global values

    if col not in columns:
        print(f"ERROR: column '{col}' not found.")
        return None, None

    col_index = columns.index(col)
    val_clean = val.strip('')

    table_name = None
    for tname, cols in tables.items():
        if cols == columns:
            table_name = tname
            break

    index_name = f"{table_name}_{col}"

    if op == "=" and index_name in indexes:
        print(f"Index used for WHERE: {index_name}")

        tree = indexes[index_name]
        row_ids = tree.search(val_clean)

        result = []
        for rid in row_ids:
            if rid in values:
                result.append(values[rid])

        return columns, result

    if op not in ['>', '<', '=']:
        print(f"ERROR: unknown operator '{op}'. Use one of: >, <, =")
        return None, None

    result = []
    for row in rows:
        cell_value = row[col_index]

        if (
            (op == '>' and cell_value > val_clean) or
            (op == '<' and cell_value < val_clean) or
            (op == '=' and cell_value == val_clean)
        ):
            result.append(row)

    return columns, result

```

```

#SELECT FROM table_name_1 [JOIN table_name_2 [ON t1_column = t2_column]] [WHERE condition];
def select(c, len_c_words):
    if ('join' in c) and ('where' in c):
        if 'on' in c:
            name, rows = join_on(c[2], c[4], True, c[6], c[8])
            columns, result = where(name, rows, c[10], c[11], c[12])
            print_table(columns, result)
        else:
            name, rows = join_on(c[2], c[4], False, None, None)
            columns, result = where(name, rows, c[6], c[7], c[8])
            print_table(columns, result)
        return

    elif 'join' in c:
        if 'on' in c:
            name, rows = join_on(c[2], c[4], True, c[6], c[8])
        else:
            name, rows = join_on(c[2], c[4], False, None, None)
        print_table(name, rows)
        return

    elif 'where' in c:
        if check_table(c[2]):
            columns, result = where(tables[c[2]], get_rows(c[2]), c[4], c[5], c[6])
            print_table(columns, result)
        return

    elif (len_c_words == 3):
        if check_table(c[2]):
            rows = get_rows(c[2])
            print_table(tables[c[2]], rows)
        return

    else:
        print('ERROR: failed to recognize command keywords.')
        print('Try: SELECT FROM table_name_1 [JOIN table_name_2 [ON t1_column = t2_column]] [WHERE condition];')
        return

```

Тепер зробимо таку ж версію, однак щоб значення зберігались у масивах замість словників.

```

tables = []
values = []
i = 0
need_index = []
indexes = {}

```

```

def find_table(table_name):
    for name, cols in tables:
        if name == table_name:
            return cols
    return None

def print_table(table_name, table_rows):
    if (table_name == None) or (table_rows == None):
        print('ERROR: Columns or rows are not found.')
        return
    print(table_name)
    print('-----')
    for row in table_rows:
        print(row)

```

```

# CREATE table (columns [INDEXED]);
def create(c):
    is_var1_correct = is_correct(c[1])
    if not is_var1_correct:
        print('ERROR: variable entered incorrectly.')
        return
    global tables
    global need_index
    for tname, _ in tables:
        if tname == c[1]:
            print(f'ERROR: cannot create "{c[1]}" again.')
            return

    columns = words_inside_brackets(c)

    if "INDEXED" in columns:
        while "INDEXED" in columns:
            idx = columns.index("INDEXED")
            col_name = columns[idx - 1]
            need_index.append(f"{c[1]}_{col_name}")
            del columns[idx]

    tables.append([c[1], columns])
    print(f'Table "{c[1]}" was created.')

```

```

def insert(c, is_into):
    if is_into:
        table_name = c[2]
    else:
        table_name = c[1]

    global tables, values, i, indexes, need_index

    cols = find_table(table_name)
    if cols is None:
        print(f'ERROR: no table with name "{table_name}".')
        return

    row_vals = words_inside_brackets(c)
    if len(row_vals) != len(cols):
        print('ERROR: number columns != number values.')
        return

    values.append([table_name, i, row_vals])
    print(f'Values were inserted into "{table_name}".')

    for col_index, col_name in enumerate(cols):
        idx_name = f"{table_name}_{col_name}"
        if idx_name in need_index:
            if idx_name not in indexes:
                indexes[idx_name] = Binary_Tree()
            indexes[idx_name].insert(row_vals[col_index], i)
            #indexes[idx_name].print_tree();

    i += 1

```

```

def get_rows(table_name):
    result = []
    for tname, row_id, vals in values:
        if tname == table_name:
            result.append(vals)
    return result

def check_table(table_name):
    return find_table(table_name) is not None

```

```

def join_on(table_name_1, table_name_2, on, column_1, column_2):
    if not (check_table(table_name_1) and check_table(table_name_2)):
        return None, None
    rows1 = get_rows(table_name_1)
    rows2 = get_rows(table_name_2)
    cols1 = find_table(table_name_1)
    cols2 = find_table(table_name_2)
    if cols1 is None or cols2 is None:
        return None, None
    if not on:
        joined_rows = [r1 + r2 for r1 in rows1 for r2 in rows2]
        return cols1 + cols2, joined_rows
    if column_1 not in cols1 or column_2 not in cols2:
        print("ERROR: column name not found.")
        return None, None
    i1 = cols1.index(column_1)
    i2 = cols2.index(column_2)
    index_name = f"{table_name_2}_{column_2}"
    global indexes
    if index_name in indexes:
        print(f"Index used for JOIN: {index_name}")
        tree = indexes[index_name]
        joined_rows = []
        for r1 in rows1:
            key = r1[i1]
            matching_ids = tree.search(key)
            for t2, rid2, r2 in values:
                if rid2 in matching_ids and t2 == table_name_2:
                    joined_rows.append(r1 + r2)
        return cols1 + cols2, joined_rows

    joined_rows = []
    for r1 in rows1:
        for r2 in rows2:
            if r1[i1] == r2[i2]:
                joined_rows.append(r1 + r2)

    return cols1 + cols2, joined_rows

```

```

def where(columns, rows, col, op, val):
    global indexes, tables, values
    if col not in columns:
        print(f"ERROR: column '{col}' not found.")
        return None, None
    col_index = columns.index(col)
    val_clean = val.strip('')
    table_name = None
    for tname, cols in tables:
        if cols == columns:
            table_name = tname
            break
    index_name = f"{table_name}_{col}"
    if op == "=" and index_name in indexes:
        print(f"Index used for WHERE: {index_name}")
        tree = indexes[index_name]
        row_ids = tree.search(val_clean)
        result = []
        for tname, rid, rowvals in values:
            if rid in row_ids and tname == table_name:
                result.append(rowvals)
        return columns, result

    if op not in ['>', '<', '=']:
        print(f"ERROR: unknown operator '{op}'. Use: > < =")
        return None, None

    result = []
    for row in rows:
        cell_value = row[col_index]

        if ((op == '>' and cell_value > val_clean) or
            (op == '<' and cell_value < val_clean) or
            (op == '=' and cell_value == val_clean)):
            result.append(row)

    return columns, result

```

```

def select(c, len_c_words):
    if ('join' in c) and ('where' in c):
        if 'on' in c:
            name, rows = join_on(c[2], c[4], True, c[6], c[8])
            columns, result = where(name, rows, c[10], c[11], c[12])
        else:
            name, rows = join_on(c[2], c[4], False, None, None)
            columns, result = where(name, rows, c[6], c[7], c[8])

        print_table(columns, result)
        return

    elif 'join' in c:
        if 'on' in c:
            name, rows = join_on(c[2], c[4], True, c[6], c[8])
        else:
            name, rows = join_on(c[2], c[4], False, None, None)

        print_table(name, rows)
        return

    elif 'where' in c:
        if check_table(c[2]):
            cols = find_table(c[2])
            rows = get_rows(c[2])
            columns, result = where(cols, rows, c[4], c[5], c[6])
            print_table(columns, result)
        return

    elif len_c_words == 3:
        if check_table(c[2]):
            cols = find_table(c[2])
            rows = get_rows(c[2])
            print_table(cols, rows)
        return

    print("ERROR: failed to recognize command keywords.")

```

Порівняємо обидва варіанти з та без індексів. (З зберіганням значень у словнику з індексами та без індексів, з зберіганням значень у масивах з індексами та без індексів)

```
import subprocess
import time
import threading

def run_program(filename, commands, log_filename):
    proc = subprocess.Popen(
        ["python", filename],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
        bufsize=1
    )
    log_file = open(log_filename, "w", encoding="utf-8")
    output_buffer = []
    def reader_thread():
        for line in proc.stdout:
            output_buffer.append(line)
            log_file.write(line)
    t = threading.Thread(target=reader_thread, daemon=True)
    t.start()
    results = {}
    for cmd_name, cmd_text in commands:
        output_buffer.clear()
        proc.stdin.write(cmd_text + "\n")
        proc.stdin.flush()
        start = time.perf_counter()
        while True:
            time.sleep(0.0005)
            if len(output_buffer) > 0:
                break
        end = time.perf_counter()
        if cmd_name.startswith("SELECT"):
            results[cmd_name] = end - start
    proc.terminate()
    log_file.close()
    return results
```

```

commands = [
    ("CREATE1",      'create cat (1, 2, 3, 4);'),
    ("INSERT11",     'insert cat (a, b, c, d);'),
    ("INSERT12",     'insert cat (a, a, e, f);'),
    ("INSERT13",     'insert cat (a, a, a, g);'),
    ("INSERT14",     'insert cat (a, a, a, a);'),
    ("INSERT15",     'insert cat (a, b, c, d);'),
    ("INSERT16",     'insert cat (a, a, e, f);'),
    ("INSERT17",     'insert cat (a, a, a, g);'),
    ("INSERT18",     'insert cat (a, a, a, a);'),

    ("CREATE2",      'create dog (5, 6, 7, 8);'),
    ("INSERT21",     'insert dog (b, b, b, b);'),
    ("INSERT22",     'insert dog (a, a, a, a);'),
    ("INSERT23",     'insert dog (i, j, k, l);'),
    ("INSERT24",     'insert dog (a, j, d, c);'),
    ("INSERT25",     'insert dog (b, b, b, b);'),
    ("INSERT26",     'insert dog (a, a, a, a);'),
    ("INSERT27",     'insert dog (i, j, k, l);'),
    ("INSERT28",     'insert dog (a, j, d, c);'),

    ("SELECT_ALL_JOIN",      "select from cat join dog;"),
    ("SELECT_JOIN_ON",       "select from cat join dog on 3 = 8;"),
    ("SELECT_WHERE_<",      "select from cat where 2 < b;"),
    ("SELECT_WHERE_=",       "select from cat where 4 = a;"),
    ("SELECT_WHERE_JOIN",    "select from cat join dog on 1 = 5 where 8 = a;"),

    ("STOP", "stop;")
]

```

```

commands_i = [
    ("CREATE1",      'create cat (1 INDEXED, 2 INDEXED, 3 INDEXED, 4 INDEXED);'),
    ("INSERT11",     'insert cat (a, b, c, d);'),
    ("INSERT12",     'insert cat (a, a, e, f);'),
    ("INSERT13",     'insert cat (a, a, a, g);'),
    ("INSERT14",     'insert cat (a, a, a, a);'),
    ("INSERT15",     'insert cat (a, b, c, d);'),
    ("INSERT16",     'insert cat (a, a, e, f);'),
    ("INSERT17",     'insert cat (a, a, a, g);'),
    ("INSERT18",     'insert cat (a, a, a, a);'),

    ("CREATE2",      'create dog (5 INDEXED, 6 INDEXED, 7 INDEXED, 8 INDEXED);'),
    ("INSERT21",     'insert dog (b, b, b, b);'),
    ("INSERT22",     'insert dog (a, a, a, a);'),
    ("INSERT23",     'insert dog (i, j, k, l);'),
    ("INSERT24",     'insert dog (a, j, d, c);'),
    ("INSERT25",     'insert dog (b, b, b, b);'),
    ("INSERT26",     'insert dog (a, a, a, a);'),
    ("INSERT27",     'insert dog (i, j, k, l);'),
    ("INSERT28",     'insert dog (a, j, d, c);'),

    ("SELECT_ALL_JOIN",      "select from cat join dog;"),
    ("SELECT_JOIN_ON",       "select from cat join dog on 3 = 8;"),
    ("SELECT_WHERE_<",      "select from cat where 2 < b;"),
    ("SELECT_WHERE_=",       "select from cat where 4 = a;"),
    ("SELECT_WHERE_JOIN",    "select from cat join dog on 1 = 5 where 8 = a;"),

    ("STOP", "stop;")
]

```

```

all_res = {}
print('NO INDEXED')
print("Running file without arrey...")
r1 = run_program("Lab3_FB-34_Tsaryk.py", commands, "log_1.txt")

print("Running file with arrey...")
r2 = run_program("Lab3_FB-34_Tsaryk_array.py", commands, "log_2.txt")

for key in r1:
    if key.startswith("SELECT"):
        print(f"{key}: no_arrey={r1[key]:.6f}s    arrey={r2[key]:.6f}s")

print("FASTER VERSION:")
t1 = sum(r1[k] for k in r1 if k.startswith("SELECT"))
t2 = sum(r2[k] for k in r2 if k.startswith("SELECT"))
print("without arrey" if t1 < t2 else "with arrey")
all_res['No_i without array'] = t1
all_res['No_i with array'] = t2

print('INDEXED')
print("Running file without arrey...")
r1i = run_program("Lab3_FB-34_Tsaryk.py", commands_i, "log_i1.txt")

print("Running file with arrey...")
r2i = run_program("Lab3_FB-34_Tsaryk_array.py", commands_i, "log_i2.txt")

for key in r1i:
    if key.startswith("SELECT"):
        print(f"{key}: no_arrey_i={r1i[key]:.6f}s    arrey_i={r2i[key]:.6f}s")

print("FASTER VERSION:")
t1i = sum(r1i[k] for k in r1i if k.startswith("SELECT"))
t2i = sum(r2i[k] for k in r2i if k.startswith("SELECT"))
print("without arrey" if t1i < t2i else "with arrey")
all_res['With_i without array'] = t1i
all_res['With_i with array'] = t2i

```

Результат:

```
In [7]: %runfile 'C:/Users/User/Desktop/Новый компьютер/КПИ/2025-26/Основы анализа алгоритмов/Lab3/test.py' --wdir
NO INDEXED
Running file without arrey...
Running file with arrey...
SELECT_ALL_JOIN: no_arrey=0.000628s    arrey=0.000596s
SELECT_JOIN_ON: no_arrey=0.000625s    arrey=0.000579s
SELECT_WHERE_<: no_arrey=0.000909s    arrey=0.000575s
SELECT_WHERE_=: no_arrey=0.000560s    arrey=0.000600s
SELECT_WHERE_JOIN: no_arrey=0.000551s    arrey=0.000641s
FASTER VERSION:
with arrey
INDEXED
Running file without arrey...
Running file with arrey...
SELECT_ALL_JOIN: no_arrey_i=0.000607s    arrey_i=0.000549s
SELECT_JOIN_ON: no_arrey_i=0.000574s    arrey_i=0.000555s
SELECT_WHERE_<: no_arrey_i=0.000566s    arrey_i=0.000553s
SELECT_WHERE_=: no_arrey_i=0.000566s    arrey_i=0.000538s
SELECT_WHERE_JOIN: no_arrey_i=0.000568s    arrey_i=0.000555s
FASTER VERSION:
with arrey
```

Бачимо що найшвидшим варіантом є код з використанням масивів і з індексами.

 log_1.txt	10.12.2025 14:15	Текстовый докум...	6 КБ
 log_2.txt	10.12.2025 14:15	Текстовый докум...	6 КБ
 log_i1.txt	10.12.2025 14:15	Текстовый докум...	6 КБ
 log_i2.txt	10.12.2025 14:15	Текстовый докум...	6 КБ

[illegible]